MEMÒRIA P2

Programació avançada i estructura de dades

G08

Nil de Creus Massot – nil.decreus

Oleguer Almuni Orra – oleguer.almuni

Laura Cuenca Plata – laura.cuenca

Alexandre Contreras Urbita -alexandre.contreras

Contingut

Llenguatge de programació escollit....2
Anàlisi d'implementacions de tercers....3
Grafs....4
Justificació de disseny....4
Explicació d'algorismes....5
Anàlisi de resultats....8
Explicació del mètodes de proves....10
Problemes observats....10
Arbres binaris....11
Justificació de disseny....11
Explicació d'algorismes....11
Anàlisi de resultats....14
Explicació del mètodes de proves....16
Problemes observats....17
Arbres-R....18
Justificació de disseny....18
Explicació d'algorismes....18
Anàlisi de resultats....6
Explicació del mètodes de proves....25
Problemes observats....25
Taules....26
Justificació de disseny....26
Explicació d'algorismes....27
Anàlisi de resultats....28
Explicació del mètodes de proves....33
Problemes observats....33

Llenguatge de programació escollit

Per a la resolució de la pràctica s'ha decidit programar el codi en Java. Aquesta elecció s'ha fet basant-nos en diferents punts. El primer punt i, potser, el més decisiu és la limitació de coneixements. El nostre ventall d'opcions estava limitat a Java o C des d'un principi. Un cop vistes les opcions que es tenien, es va decidir que utilitzar Java seria la millor decisió.

Es va prendre el criteri d'utilitzar un llenguatge que fos més intuïtiu i còmode de programar, i no d'escollir el que permetés implementar els algorismes aconseguint costos més baixos. Java compta amb una gran comunitat i documentació oficial, permetent que en cas de que sorgissin dubtes poder aconseguir informació de forma més fiable i senzilla a través d'internet. Apart, Java te de per si una gran varietat de llibreries que faciliten molt les tasques com podria ser la lectura de fitxers i la seva càrrega en el programa, cosa que en C hagués sigut més complicat. També és important tenir en consideració que el llenguatge de Java té una sintaxi clara i estructurada, fet que facilita la lectura del codi.

Anàlisi d’implementacions de tercers

En aquest programa, s'utilitzen diverses llibreries de Java per gestionar les dades i les operacions necessàries per desenvolupar les diferents funcionalitats. S'utilitza la llibreria java.awt.*, que inclou classes com `Color`. Aquestes classe té un cost computacional baix, ja que representa estructures de dades simples. `Color` s'utilitza per representar i manipular colors en Java.

També es fa servir ArrayList de Java, una implementació de llista dinàmica que permet emmagatzemar elements i accedir a ells de manera eficient. Les operacions típiques d'aquesta classe tenen un cost computacional de O(1) per accés i inserció (amortitzat) i O(n) en el pitjor dels casos per eliminació.

Per a implementar cues, hem emprat també LinkedList de Java. Tot i que el cost de cerca pot ser O(n) en el pitjor dels casos, les funcions que hem utilitzat d’aquesta implementació de llista no corren aquest risc, ja que ens hem limitat a utilitzar poll() i add() durant la implementació de la funcionalitat de la cerca d’infiltrats dins el canal sucre (arbres binaris). Aquestes dues funcions tenen cost O(1).

Una altra llibreria utilitzada és PriorityQueue també de Java, una cua amb prioritat que permet accedir a l'element amb la prioritat més alta de manera eficient, amb operacions d'inserció i eliminació de l’últim element amb un cost de O(log n) i accés a aquest amb un cost de O(1).

En general, les funcions i implementacions que hem utilitzat de les llibreries de Java no tenen un cost gaire elevat, però és cert que tot son mètodes i classes que podríem haver implementat nosaltres i estalviar així importar llibreries plenes de codi que no utilitzem durant la pràctica.

Grafs

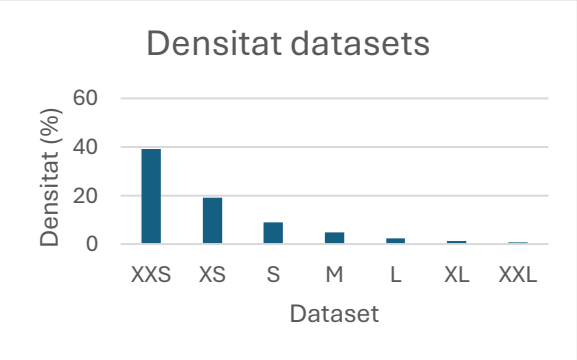
Justificació de disseny

Per a la representació del graf s'ha vist més adient implementar una llista que una matriu d'adjacències. El raonament darrera d'aquesta decisió s'ha basat en l'estudi de la densitat dels datasets i la comparació de costos de dues casuístiques; donat dos vèrtex determinar si un és adjacent a l'altre i, donat un vèrtex conèixer tots aquells vèrtex que són adjacents a aquest.

Representació	Consultar adjacència de u i	Recórrer adjacents de v
Llista d'adjacències	$O(d(u))$ o $O(d(v))$	$\theta(d(v))$
Matriu d'adjacències	$\theta(1)$	$\theta(\|V\|)$

Com es pot observar a la taula, els costos de la llista d'adjacències són directament proporcionals al grau dels vèrtex que es consulten. Per tant, si la densitat dels datasets és baixa, la implementació d'una llista resultarà més eficient tenint en compte que amb una matriu el cost de recórrer adjacents d'un vèrtex té cost $\theta(|v|)$.

Per a trobar els percentatges de densitat de cada dataset, s'ha dividit el nombre rutes entre el nombre màxim de connexions possibles. Per a determinar les màximes connexions s'ha aplicat la següent fórmula: $\frac{n(n-1)}{2}$ on 'n' són el total de seus.



Dataset	N vèrtex	N rutes
XXS	8	23
XS	16	33
S	32	88
M	64	175
L	128	357
XL	256	695
XXL	512	1466

Com es pot observar a la gràfica, quant major és el volum dels datasets menor és el grau de densitat que presenten. Per línia general, excepte pels datasets de mida XXS i XS els resultats reflecteixen un nivell de densitat baix pel que es pot concloure que la implementació d'una llista és l'opció més adient. Per tant, tot i que el cost de consultar una adjacència serà major que amb una matriu, degut a que els grafs són prou dispersos aquest cost no resultarà excessivament alt. D'altra banda, respecte l'ús d'una matriu el cost de recórrer adjacents resultarà significativament menor.

Per a emmagatzemar els nodes adjacents a cada seu, s'ha decidit utilitzar ArrayLists. Aquesta decisió s'ha pres tenint en compte que tant en recorreguts com ordenacions topològiques s'hauran de fer moltes consultes i el cost d'accés a un Arraylist és directe. En canvi amb una linkedlist, en el pitjor cas el cost d'accés és d'n. Cal remarcar que aquest cost no seria gaire elevat ja que com s'ha vist els grafs presenten una densitat baixa.

Un possible inconvenient d'utilitzar Arraylist respecte Linkedlist és que si la llista és plena el cost d'inserció és d'n. En canvi, amb una Linkedlist el cost d'inserció sempre és 1. De

totes maneres, sobrepassar la capacitat d'una llista d'adjacents no serà habitual degut al elevat grau de dispersió dels grafs.

Explicació d'algorismes

Dijkstra

Per a la implementació del Dijkstra, inicialment es va optar per desar la informació de camins i distàncies a arrays d'enters. Això suposava un cost excessivament alt a l'hora d'accedir i modificar la informació del node actual a cadascun d'aquests arrays. Per aquest motiu es va decidir plantejar un altre enfoc. En aquets enfoc alternatiu, la informació de distància i node pare es va establir com a atribut de la classe "Seu", obtenint un cost d'accés directe.

Per a la gestió de restriccions, en avaluar cada adjacent del node actual es verifica si la ruta que els connecta està restringida. En cas de no estar-hi, es comprova si l'adjacent està visitat i si no ho està s'avalua la nova distància. En canvi, si la ruta està restringida, s'agafa el següent adjacent.

Si totes les rutes que uneixen el node pare amb els respectius nodes adjacents estan restringides, es llança una excepció per a indicar que el camí entre les seus sol·licitades no és viable.

Un escenari que no es va tenir en consideració i que va donar problemes a l'hora d'executar el codi és el cas en que no hi ha cap camí entre les seus especificades.

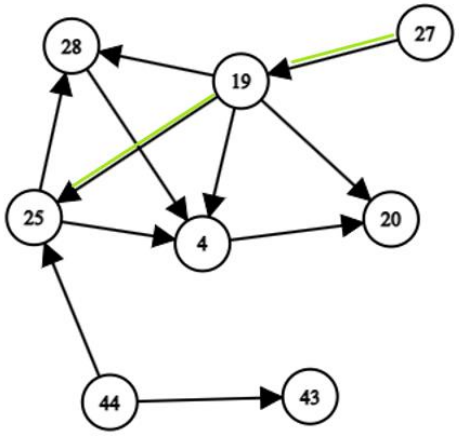


Figura 1 Representació gràfica del Dataset XXS

Per exemple, prenent com a referència el graf de la figura 1, si es volgués anar de la seu 27 a la seu 43, la direccionalitat de les rutes no ho permetria. En conseqüència, el node pare de la seu 44 no s'arribaria a actualitzar mai i romandria a valor -1 (valor d'inicialització). En el nostre codi, això suposava un problema ja que a l'hora al reconstruir el camí s'utilitzava una cerca binaria per a determinar l'índex del pare del node actual a la llista de seus a partir

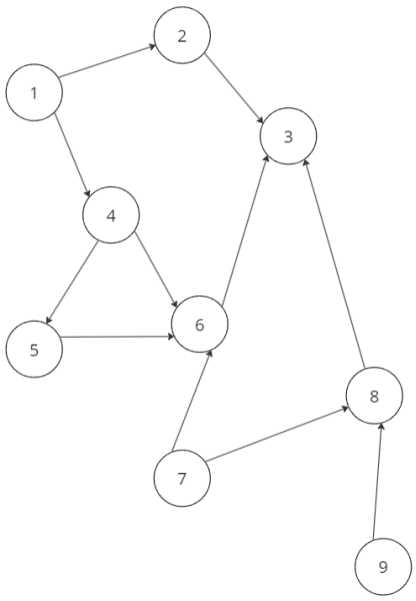
del seu ID. Com no existeix cap seu amb un número d'identificador negatiu es feien crides recursives fins a fer overflow.

La solució davant d'aquest problema va ser llançar una excepció en cas que en reconstruir el camí final es trobés un node pare amb identificador negatiu.

La part positiva d'haver detectat aquest error, és que es va plantejar un enfoc més eficient per a obtenir el pare d'un node. Concretament, es va modificar el mètode que retornava l'ID del pare d'una seu per a que retornés un objecte "Seu". Per tant, ja no era necessari recórrer a la cerca binaria per trobar l'índex associat a l'ID retornat, ja que es disposava de l'objecte en sí. D'aquesta manera, el cost d'assignar un nou node actual al mètode de reconstrucció resultava directe en comptes del cost $O(log\, N)$ de la cerca binaria.

DFS

A l'hora d'implementar la intel·ligència comercial hi han hagut varius dubtes sobre com fer el recorregut per obtenir les seus en l'ordre que ens interessa. No quedava clar com es volia representar l'ordre de les fases més avançades de la cadena de producció. Al recórrer l'arbre hi havia varies maneres d'obtenir aquest ordre, i decidir entre varius criteris.



Il·lustració 2:Representació de Graf.

En aquest exemple de representació es pot veure que el node 6 depèn del 4 i el 5 però el 5 del 4. Aquí es pot veure un clar exemple d'aquests diferents criteris per escollir aquest ordre de cadena. Per un costat podem considerar que el 5 va abans que el 6 però també es podria considerar que el 5 i el 6 estan al mateix nivell.

En aquest funcionalitat es veu certa similitud amb la cadena de producció, però com es pot veure en l'exemple es poden plantejar altres criteris i així resoldre-ho amb un algorisme diferent al toposort, ja que es pot mirar de fer un recorregut agafant cert criteri en comptes de fer una ordenació de dependències. Això en deixa dos opcions clares, el BFS i el DFS.

El problema d'aplicar una recorregut DFS es que recorre en profunditat cada vèrtex, el que dificulta obtenir una representació de fases més a menys avançades a nivell global. Aquesta seria una bona opció per fer aquesta representació de cada branca.

En el cas contrari trobem el BFS el qual ens permet fer passades per nivell que es justament el que interessa. El problema es quan el graf te diferents nodes pare, aquest cas dificulta aconseguir aquesta representació global de les fases i el seu estat. Com es pot veure en l'exemple de la figura 2 això seria el cas del node 1, 7, 9. El node 1 quedaria per sota del node 8 en l'ordre obtingut.

Es pot veure que els dos algorismes tenen els seus inconvenients. Per solucionar-los s'ha implementat un arraylist N, on N es el nivell al que s'està accedint, que contingui altres arraylists on es guardaran els nodes del nivell actual. Partint d'aquest concepte es tindrà que anar accedint als diferents nivells del graf i guardant en la posició corresponent. S'ha optat per aplicar el DFS ja que s'ha vist més intuïtiu fer les baixades verticals, és a dir, baixant primer en profunditat i així anar incrementant el nivell al que s'està.

Aquest algorisme es basa en fer recorreguts en profunditat de forma recursiva expandint cada branca fins al final com ve s'ha comentat anteriorment. Per tal de portar el comptatge sobre quin nivell estem, és a dir, el grau de profunditat en el que ens trobem el fet que sigui un algorisme recursiu ens permet anar passant a quin nivell accedim passant-lo com a paràmetre de forma incremental. Sempre es passarà el pròxim nivell que accedim fent que les baixades recursives expressin cada nivell.

Execució de la cadena - Toposort

Per a resoldre l'ordre d'execució s'ha escollit un algorisme d'ordenació topològica. Aquests algorismes són utilitzat per resoldre dependències unidireccionals entre elements. En el cas de les seus, una no pot començar a produir fins que la seu que li proveeix hagi acabat el seu procés de producció, per lo qual aquest algorisme resol el problema plantejat si volem obtenir l'ordre de les seus adequat per al correcte funcionament de la cadena de producció.

Aquest algorisme es basa en la cerca per profunditat DFS, que avança per una branca tot lo possible abans de recular, arribant així a una possible seu final de la cadena de producció. No obstant això ens deixaria amb un ordre invers al que volem obtenir, ja que l'ordre desitjat es el que resol les dependències per poder seguir avançant. Això es resol fàcilment afegint els elements en una pila en comptes d'una llista.

No obstant l'informació presentada a l'anterior paràgraf, s'ha decidit utilitzar una Queue (implementada com LinkedList) per a guardar l'ordre de dependències de les seus per a una gestió FIFO. Això es deu a la nostra gestió de recorregut, es recorren els vèrtex en direcció inversa a les arestes, és a dir, mirem els proveïdors de cada seu i no a qui proveeixen. D'aquesta manera, en arribar a una seu sense proveïdors, és a dir, sense dependències, aquesta s'afegeix primer, assegurant així l'ordre correcte dins la cadena de producció. A més, el cost de les operacions *add* i *remove* quan no és a una posició específica és constant O(1) i això ens permet una bona gestió dels recursos.

Un cop acabada la implementació ens vam adonar de que s'estava obviant un aspecte clau en la utilització d'aquest algorisme, la ciclicitat dels grafs sotmesos a aquesta ordenació.

Els grafs cíclics representen una problemàtica per a l'algorisme toposort, ja que pot resultar en un bucle infinit. Per això, més endavant, es va implementar un mètode que, mitjançant la cerca en profunditat (DFS), intenta identificar cicles dins del graf. Aquesta estratègia funciona de manera que, per a cada seu no visitada, es crida la funció de recorregut DFS. Aquesta funció marca la seu actual com a visitada i, per a cada node adjacent, comprova si ja havia estat visitat en aquest recorregut o, en cas contrari, realitza la crida recursiva.

Anàlisi de resultats

Com s'ha comentat, per tal d'optimitzar la cerca de l'índex d'un node, s'ha aplicat un quicksort en base als identificadors de les seus. D'aquesta manera, sacrifiquem temps d'execució una vegada construït el graf ja que l'ordenem amb l'algorisme quicksort (temps mitjà nlogn), però guanyem eficiència en la cerca binaria. Tenint en compte que la cerca binaria s'utilitza amb freqüència per a la implementació de les diverses funcionalitats i, el quicksort únicament es fa inicialment, s'ha considerat un bon enfoc aplicar l'ordenació. Alternativament, el cost de cerca en el pitjor cas resultaria d'N ja que caldria recorre tots els nodes del graf.

Però per a comprovar si realment això suposa una reducció del cost d'execució s'analitzarà el cost del dijkstra amb els dos enfocs esmentats. La implementació de dijkstra utilitza la cerca binaria a l'hora de retreure l'índex de les seus inicials i finals especificades per l'usuari i, a l'hora de reconstruir el camí més curt.

Inicialment, la idea era comparar el pitjor cas per tots els datasets. Tenint en compte que el dijkstra s'atura una vegada visita el node destí, per tal que els costos dels datasets siguin coherents amb la seva mida, l'únic anàlisi viable era analitzar el pitjor cas. Així doncs, s'havia implementat un mètode per trobar el node inicial i final de cada dataset amb l'objectiu de calcular-ne el cost. De totes maneres, s'ha vist que això suposa varies complicacions ja que en molts casos les excepcions personalitzades aturen l'execució de l'algorisme ja que el camí entre els punts inicial i final està restringit o simplement les rutes no connecten amb la direccionalitat apropiada. Per tant, realment la càrrega d'execució no sempre és proporcional a la mida dels datasets. Per aquest motiu fer una comparativa de costos amb les diferents mides de datasets no donaria resultats coherents.

Així doncs, s'ha decidit analitzar la ruta més ràpida entre les seus 3123 i 719 (dataset XXL) i s'han obtingut els següents resultats:

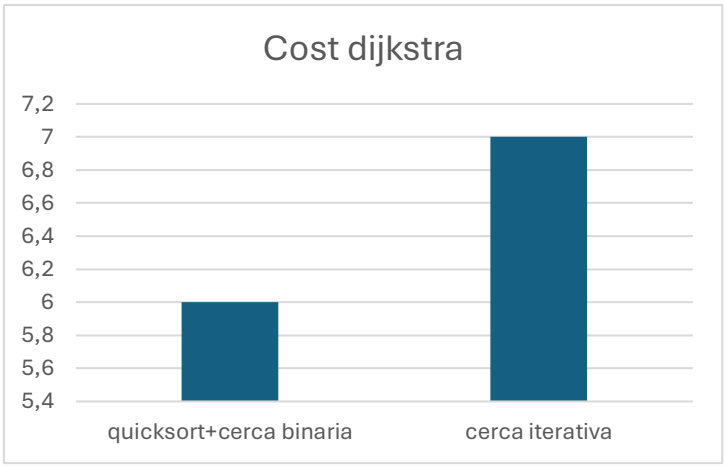


Figura 3: Cost Dijkstra

Com s'observa, l'enfoc de quicksort i cerca binaria resulta més eficient que parcejar els nodes del graph. Tot i així cal remarcar que en al alguna ocasió el cost en milisegons dels dos enfocs ha estat el mateix. Com es va veure al primer semestre l'eficiència d'un algorisme depèn de molts altres factors però de totes maneres si en ocasions, encara que sigui amb baixa freqüència, el cost obtingut és el mateix significa que la millora obtinguda no és tan significativa com esperàvem.

Per a fer una comparativa més representativa i de més pes es compararà el cost de llegir rutes per cadascun dels datasets disponibles. Al llegir rutes, per cada ruta s'ha de fer una cerca per trobar els índex dels identificadors de les dues seus llegides al graph construït fins al moment. Així doncs, s'han obtingut els següents resultats.

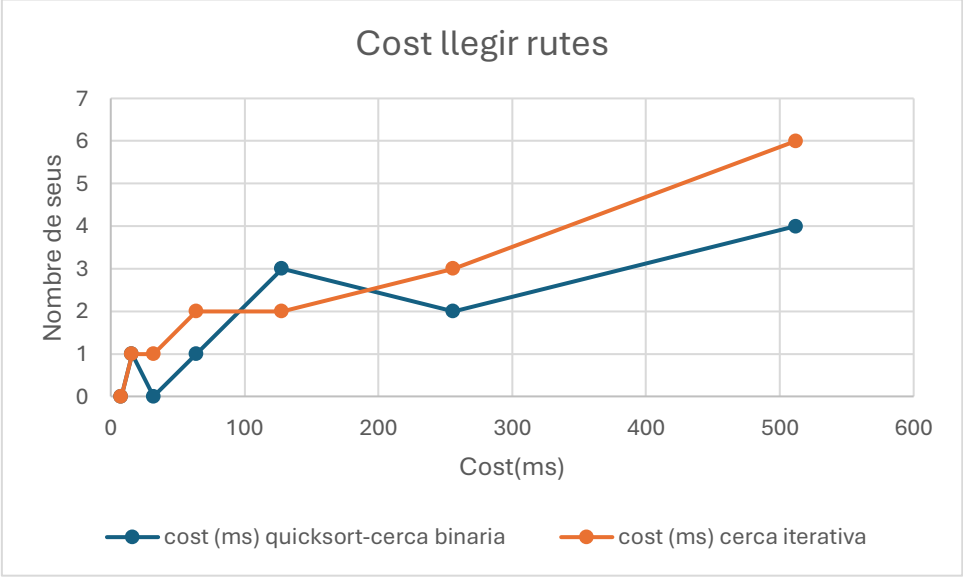


Figura 4:Cost de la lectura de rutes

Com a mida d'elements s'ha agafat el nombre de seus de cada dataset. Això és degut a que per moltes rutes que hi hagi, l'identificador a cercar es trobarà a una d'aquestes N seus. Com s'observa a la gràfica per a datasets petits sembla que no hi ha un comportament clar

que permeti extreure conclusions fermes. Això és degut a que estem parlant de dataset amb molt pocs vèrtex, pel que el pitjor cas de recórrer el graph sencer no suposa un cost elevat. En canvi haver de fer un quicksort i cerca binaria pot ser lleugerament excessiu. En canvi, per datasets amb un nombre 'elements més elevat si que s'observa com fer una cerca iterativa resulta més costós.

En conclusió l'eficiència de l'enfoc quicksort-cerca binaria resulta positiva amb datasets on l'ordre d'element és prou elevat.

Explicació dels mètodes de proves

Per tal de comprovar els algorismes implementats s'ha utilitzat una eina de representació gràfica de grafs. D'aquesta manera, per a la comprovació del dijkstra s'han pogut analitzar tant casos viables com no viables i poder gestionar les excepcions pertinents. Així doncs, s'han analitzat casos on la seu inicial i final es troben connectades i casos on no ho són.

Problemes observats

Els principals problemes observats han estat amb el dijkstra, concretament amb la gestió d'excepcions a l'hora de fer testing. Com s'ha explicat a la secció de mètode proves es va utilitzar una eina de representació gràfica per reconèixer casos viables i no viables. Aquesta eina, però, no es va utilitzar des d'un inici. Com a conseqüència, a les primeres proves de testing s'introduïen seus inicials i finals aleatòries sense considerar si estaven connectades per una ruta viable. Això suposava que el pare dijkstra d'alguns nodes quedés a -1 i la seu no fos trobada al fer la cerca binaria al reconstruir el camí més curt. Una vegada identificat el problema es va crear una excepció que en cas que el pare d'algun vèrtex fos '-1' en acabar l'algorisme dijkstra, indicaria que la ruta entre les seus especificades no existeix.

Arbres binaris

Justificació de disseny

Per a fer la representació del arbre binari format pels habitants del regne, disposàvem de dues opcions, l'explicita i la implícita. Finalment, hem optat per representar l'arbre de forma explicita, degut a que mirant les funcionalitats que havíem d'implementar en aquest apartat de la pràctica, hem vist que havíem de poder afegir i eliminar nodes, així com referenciar el pare d'un node en diversos punts. Aquests requeriments, han fet que descartéssim la representació implícita ja que resulta molt més complicat mantenir l'arbre si està en format d'array degut a que, per exemple, eliminar un node suposa tornar a endreçar tot l'arbre, mentre que en explicita només s'han de canviar els nodes a on referencien els punters afectats per el canvi.

La garantia que en tot moment mantindrem l'arbre equilibrat podria haver estat incentiu per escollit la representació implícita, doncs vol dir que es redueix considerablement el nombre d'espais null que poden haver-hi, però cal recordar que justament per a mantenir l'arbre equilibrat s'han de realitzar rotacions i aquestes son operacions on la representació explícita resulta més optima al no haver de modificar tota la representació en fer-les.

Com hem esmentat, hem afegit un punter que apunti al pare dins de cada node que ens serveix per a poder implementar la eliminació de nodes, les rotacions i una funció per trobar a quin nivell es troba cada node que s'utilitza en la cerca d'infiltrats. Si aquestes operacions no fossin necessàries, ens hauríem estalviat aquesta referència.

El criteri que hem escollit per ordenar l'arbre ha estat l'índex de sucre dels habitants. Hem pres aquesta decisió ja que era l'atribut de cada habitat pel qual calia orientar-se en més funcionalitats de les requerides. Tant en la cerca d'infiltrats com en la captura de clons, és necessari buscar dins l'arbre o bé un índex de sucre concret o bé un rang d'índex de sucre. En altres parts del canal s'utilitzen altres criteris de cerca, com en l'eliminació de nodes que s'utilitza l'id dels habitants o en la infecció d'habitants que s'ha de buscar un nom d'habitant, però en les dues funcionalitats on es cerca per índex de sucre el fet que l'arbre estigui endreçat mitjanant aquest paràmetre té altres avantatges que es s'explicaran en els seus corresponents apartats.

Explicació d'algorismes

Funcionalitats bàsiques

Afegir habitants: La implementació de la inserció de nodes que hem fet només té la particularitat que en trobar un valor igual a un altre dins l'arbre, sempre l'afegirà en algun espai a l'esquerra del que ja estava a l'arbre. D'aquesta manera si hem de buscar aquest valor, com en el cas de la captura de clons, tenim la possibilitat de no cercar a la branca dreta.

Eliminar habitants: En l'eliminació dels nodes ens podem trobar 3 escenaris diferents:

1. Cas en el que no es te cap fill on es pot eliminar directament el node.
2. Cas on només te un únic fill on el node fill passaria a ocupar la posició del pare que ha quedat buida.
3. Cas en el que un node te dos fills. En aquest escenari es busca el node fulla que estigui més a la dreta dels fills esquerra del node a eliminar per inserir com a fill dret el. En aquest escenari es substitueix el node a eliminar per el seu fill esquerra i s'introdueix el fill dret del node eliminat com a fill del node més a la dreta del que substitueix a l'eliminat.

Per tal d'auto-equilibrar l'arbre i aconseguir que les branques no quedin desproporcionades s'ha implementat un arbre AVL. Aquesta estratègia es basa en mantenir un factor d'equilibri entre 0, 1 i -1.

Per calcular aquest factor d'equilibri, s'ha anat baixant recursivament per cada branca de cada node i calculant el valor de l'altura en cada branca i quedant-se amb la major de les dos. Un cop es tenien les altures de cada un dels fills d'un node es restaven entre elles per així obtenir el factor de balanceig. Aquest factor d'equilibri s'ha anat calculant en cada moment que es necessites. En els casos en que sortíem d'aquest rang d'equilibri s'han aplicat les rotacions necessàries per tal de aconseguir els factors correctes.

Per lo tant el procés de balanceig de l'arbre seguiria el següent procés:

1. Mirar si hi ha algun node no balancejat.
2. Executar el les rotacions per balancejat en cas de no estar-ho.
    a. Mirar els factors de balanceig i escollir quina combinació de rotacions executar.
    b. Podem trobar: Esquerra-Esquerra, Esquerra-Dreta, Dreta-Esquerra o Dreta-Dreta.
3. Si esta balancejat sortir del bucle.

Aquest algorisme s'ha implementat de forma iterativa com es pot intuir amb el punt 3 del procés on s'executa un while fins que l'arbre estigui balancejat.

Cerca d'infiltrats

Per a fer la cerca d'infiltrats primer cal fer una cerca a l'arbre on es busca un habitant que tingui el mateix índex de sucre que ha introduït l'usuari. Això s'ha implementat amb una cerca recursiva que no té cap particularitat a part de que retorna null si no hi ha cap habitant que compleixi aquesta condició. Aquesta cerca té un cost logarítmic.

Si la base de dolç que ha introduït l'usuari és profiterola, s'han de buscar aquells habitants que estiguin al mateix nivell de l'arbre que l'habitant sospitós i això s'ha implementat amb una cerca per nivells de profunditat. Aquesta cerca empra una cua de prioritat utilitzant una Linked List per a recórrer l'arbre per nivells i d'aquesta manera no arriba als nodes fulla si no son aquests els nodes que comparteixen nivell amb el sospitós, estalviant així cercar parts del arbre innecessàries en tant que al tenir l'arbre una forma triangular, recórrer per nivells resultarà sempre més òptim si no s'han de recórrer tots els nodes doncs el nivell amb més nodes és l'últim. Tot seguit es busca el nivell del sospitós inicial utilitzant una

crida recursiva que en busca el pare i suma 1 a un comptador i seguidament repeteix el procés amb el pare trobat. Un cop trobat el nivell a cercar, comencem per l'arrel i afegim els seus dos fills a la cua, així com a una llista on es guarden tots els habitants que comparteixen nivell. A cada iteració es comprova si ja s'ha exhaurit el nivell mitjançant una comparació entre la quantitat d'habitants que s'han afegit d'un nivell i el nombre màxim d'habitants dins aquell nivell. Per a poder fer aquesta comparació s'afegeixen habitants null en cas que un habitant no tingui fill esquerra o dret. Evidentment, això ocupa més memòria que no si ens limitéssim a utilitzar només els habitants, però és necessari per tal de fer la cerca per nivells, que té un cost de temps prou millor sempre i quan l'habitant a cercar no es trobi en el darrer nivell.

Quan s'ha exhaurit un nivell, si es tracta del nivell que buscàvem es retorna un array amb els seus habitants, en cas contrari, es buida la llista i es segueix buscant.

D'altra banda si la base de dolç és polvoró cal mirar tots els habitants que pengin del sub-arbre del qual el sospitós n'és l'arrel i mostrar-los ordenats per el seu índex de sucre. Gràcies a que l'arbre ja està ordenat per índex de sucre, simplement cal realitzar un recorregut d'inordre partint del sospitós ja que d'aquesta manera ja es recorreran els nodes per ordre d'índex de sucre i no haurem de utilitzar un algoritme d'ordenació a posteriori, que incrementaria el cost.

Captura de clons

Per a la implementació de la captura de clons s'ha utilitzat novament el recorregut inorder, aquest cop amb la diferència que en operar en un rang d'índex de sucre, es fa una poda basada en si l'habitant està per sobre el mínim índex de sucre en el cas del fill esquerra i el màxim en el cas del fill dret a cada crida recursiva. Si l'habitant es troba dins el rang que cerquem, s'afegeix a una llista. Com l'arbre està ordenat per índex de sucre, fer el recorregut d'aquesta manera ens estalvia haver d'ordenar la llista a posteriori, simplement hem de mostrar per pantalla els habitants en l'ordre contrari a com s'han afegit a la llista, ja que l'enunciat especificava que estiguessin ordenats de major a menor.

Tree visualization

La implementació inicial de la visualització de l'arbre binari va començar amb un enfoc on la posició de l'arrel no era centralitzada; primer es mostrava l'arrel, després el subarbre dret i finalment el subarbre esquerre. Aquesta representació inicial va ser revisada durant una revisió un *checkpoint*, on es va concloure que la representació no era adequada perquè no reflectia clarament l'estructura de l'arbre, sobretot en termes de balanç.

En resposta a aquest feedback, es va reformular l'aproximació de representació. En la nova versió, vam adoptar un recorregut de l'arbre prioritizant els nodes drets (*inorder*), però amb una implementació que inicialment va resultar massa complicada. El sistema d'impressió de les branques es basava en múltiples condicions com si un node era fill esquerre o dret, la posició del pare, si estàvem al sub-arbre esquerre o dret de *root*. Aquesta complexitat va resultar en una representació que no era del tot clara, especialment en arbres grans o desequilibrats.

Després de diversos ajustos i amb l'aportació d'un dels becaris, vam simplificar l'enfocament utilitzant una metodologia basada en una llista de valors booleans. Aquesta llista registrava la direcció de cada moviment (dret o esquerre) com a veritable o fals. Aquest registre permetia, a l'hora de visualitzar cada node, recórrer la llista de booleans i aplicar una lògica simplificada per determinar la impressió de branques i indentacions: si el pas anterior diferia de l'actual, s'afegia una branca amb indentació, en cas contrari, s'aplicava només la indentació.

Simulació d'infecció

La simulació d'infecció es basa en un recorregut des d'un node origen expandint-se per tots els seus fills amb la condició que si en un punt un dels fills no s'infecta aturarà la infecció per aquella branca. Això es un punt clau per escollir un algorisme per recorre l'arbre fent un perfecte candidat el preordre el qual visita sempre el para abans que els seus fills permeten fer la proba d'infecció a abans d'expandir el recorregut cap abaix i permeten així aturar l'expansió per aquella branca en cas que no s'hagi infectat.

Un punt que pot sobtar en la implementació de l'algorisme es el fet de no haver fet un bucle per fer les crides recursives de cada node. Això es deu a que en la implementació es va fer cada fill com un atribut per separat y no com un array.

Simulació de desinfecció

Per a desenvolupar la funcionalitat de desinfecció, s'ha codificat un mètode recursiu que implementa un recorregut postordre per comptabilitzar el nombre total de nodes infectats i desinfectar-los en cas d'estar-ho. A cada crida recursiva s'avalua primerament el nombre d'infectats al subarbre esquerre del node i se suma 1 en cas que el propi node sigui llimona. Una vegada recorregut el subarbre esquerre i comptabilitzats el nombre d'infectats, s'avalua el nombre d'infectats al subarbre dret partint d'aquest valor. Així doncs, a la pujada recursiva es van acumulant els valors parcials d'infectats fins a obtenir-ne el total.

Anàlisi de resultats

Un cop realitzades totes les funcionalitats del mòdul d'arbres binaris cal analitzar aquelles qüestions que hem trobat rellevants durant el desenvolupament.

Primer de tot ens volem assegurar que els costos de les funcions bàsiques implementades són realment de cost $\theta(\log n)$ tenint en compte que l'arbre és AVL i per tant queda equilibrat.

Per a realitzar això hem calculat els costos d'afegir i eliminar un habitant per a diferents mides de dataset, per assegurar-nos que els costos son els esperats. Hem canviat la funció d'eliminar per a que utilitzes la cerca ordenada que no es el que demana la funcionalitat, però si utilitzéssim la no ordenada les comparacions no serien concloents. A continuació hi tenim els resultats:

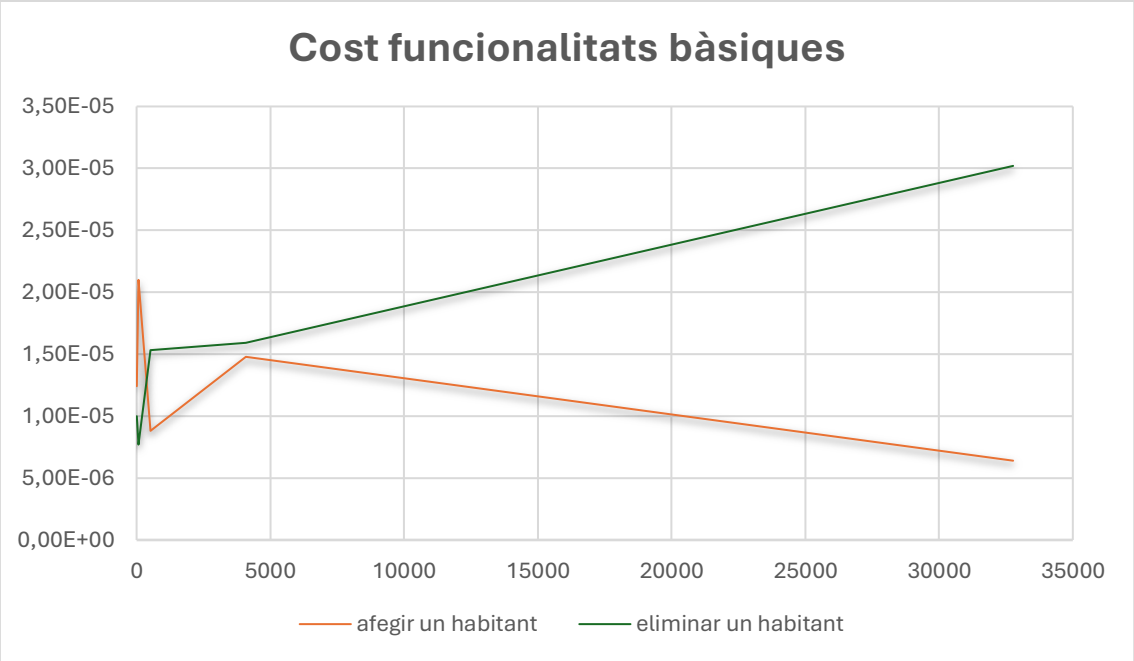


Figura 5: Cost funcionalitats bàsiques

Com podem veure a la gràfica, els costos són tant baixos amb aquests datasets que no podem observar si realment el cost d'afegir o eliminar un habitant és o no $\theta(\log n)$.

Realitzant aquesta prova ens hem adonat d'una optimització que no havíem tingut en compte durant la implementació del codi. Teníem la hipòtesi que la manera òptima de equilibrar l'arbre era anar mirant si calia equilibrar-lo cada cop que s'afegia un element durant la lectura de les dades. Pensàvem que era més ràpid arreglar petits desequilibris que no un de sol però molt gran. En fer els càlculs hem vist que estàvem equivocats doncs amb el nostre mètode construir l'arbre era 7,5 vegades més lent que si simplement l'equilibràvem al final. Per tant, podem deduir que el cost d'equilibrar no està definit per la magnitud del desequilibri.

Una de les decisions que hem pres en fer el mòdul de la pràctica que es centra en els arbres binaris de cerca ha estat per a quin criteri de la classe Citizen ordenàvem l'arbre. En l'enunciat, es citen tres atributs amb els que s'hauran de realitzar cerca, l'id per a eliminar a un habitant, l'índex de sucre per a fer la cerca d'infiltrats i la captura de clons i finalment el nom de l'habitant per a realitzar la simulació d'infecció.

Com hem comentat anteriorment, ens hem decantat per ordenar per índex de sucre, doncs les funcions que més s'ajusten a l'ús habitual d'un arbre binari de cerca són les que s'afavoreixen més de que l'arbre estigui ordenat segons aquest criteri. Tot-i-així, durant l'execució del programa haurem de fer cerques per id i per nom, que no tindran el benefici de poder recórrer l'arbre de forma ordenada i doncs ens sembla interessant veure quina diferència de cost hi ha entre fer una cerca amb l'arbre ordenat i fer-ne un altre amb l'arbre desordenat o ordenat per un criteri que no és el mateix que el de la cerca.

Per a fer aquest anàlisi hem abstret les funcions de cerca de les funcionalitats que tenen al codi per a poder visualitzar únicament el cost de la cerca en sí. Hem utilitzat el dataset de mida mitja i hem calculat els temps de cerca de una funció que seguia l'ordre del arbre i un altre que no el seguia, per a trobar un dels nodes fulla del arbre, el mateix per a les dues cerques. A continuació tenim els resultats de la mitjana de 5 mostres (el temps està en segons):

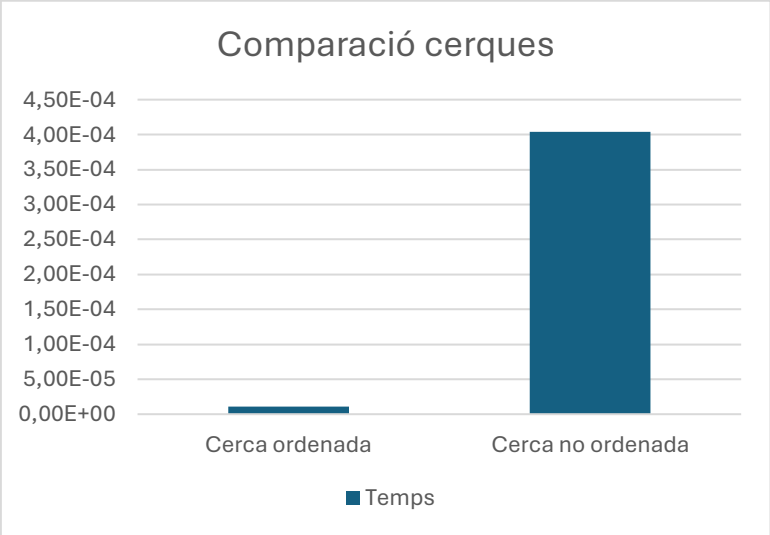


Figura 6: Comparació entre cerca ordenada i no ordenada

Com es pot observar, la diferència entre recórrer l'arbre seguint l'ordre amb el que ha estat construït i fer-ho per un altre criteri és prou significativa, doncs amb l'entorn amb el que hem fet les proves, és 10 vegades més ràpid recórrer l'arbre de forma ordenada.

Cal tenir en compte també que l'habitant que hem cercat estava cap al mig de l'arbre. En el cas de la cerca ordenada això no influeix gaire, ja que va cercant a l'esquerra o dreta del node actual en funció del valor que busca, però en el cas de la cerca no ordenada si l'habitant hagués estat cap al final de l'arbre, és a dir a la punta dreta, el cost temporal s'hagués duplicat ja que les cerques no ordenades recorren l'arbre de principi a fi.

Explicació dels mètodes de proves

Fins que no es va implementar la visualització de l'arbre el mètode de proves consistia en analitzar amb l'eina de *debugging* com els diferents algorismes implementats afectaven l'estructura de l'arbre. Òbviament, aquest enfoc presentava una limitació evident de la mida del dataset a analitzar. Més endavant, la implementació de la funcionalitat de visualització va permetre comprovar els algorismes amb volums de dades majors.

Comprovar si les funcionalitats estaven ben implementades ha estat fàcil en aquest mòdul, doncs al tenir la representació visual de l'arbre binari podíem veure gràficament si el que fèiem realment funcionava.

Realment ha estat la representació gràfica la funcionalitat que hem hagut de comprovar amb més cura, però com l'arbre es ordenat per índex de sucre dels habitants, només hem hagut de mirar si l'ordre tenia sentit en diferents datasets. Un cop hem vist que la representació era correcte hem pogut fer les comprovacions de les demés funcionalitats.

En el cas d'afegir i eliminar senzillament miràvem a la representació si s'hi veien reflectits els canvis fets en l'arbre.

Per a la cerca d'infiltrats hem mirat que es fes bé la cerca per nivells mirant en la representació que tots els habitants trobats pertanyien o bé al mateix nivell dins l'arbre o bé penjaven del subarbre adient depenent de si la base de dolç introduïda era profiterola o polvoró.

Amb la captura de clons ha resultat similar, només havíem de mirar que en la representació hi havia dins el rang introduït tots aquells habitants llistats per la funció, que no en faltava cap i que estaven en ordre.

I finalment simulació d'infecció i desinfecció es tractava de mirar si els colors dels habitants canviaven en funció de si estaven infectats o no.

Problemes observats

Per a implementar les rotacions simples, una de les consideracions que inicialment no es van tenir en compte és que cada node té una referència al seu pare. D'aquesta manera, cada moviment té una doble direccionalitat a actualitzar; la pare-fill i fill-pare. Degut a que inicialment no s'actualitzaven les referències *up-link,* l'arbre no es balancejava correctament.

També, a l'hora de fer l'equilibri encara no havíem implementat la visualització i inicialment vam donar per bona una versió de la funció d'equilibri que no funcionava de forma completament correcte. Un cop vam poder visualitzar l'arbre, ens vam adonar que els desequilibris de més de 2 no es gestionaven correctament i ho vam corregir.

La visualització ens va donar problemes quan la vam refer, com s'ha comentat en el seu apartat específic, doncs la implementació inicial tenia masses condicions i feia que mantenir-lo i arreglar-lo fos complicat.

A les demès funcionalitats no hi vam trobar cap problema o obstacle, com a molt en la cerca d'infiltrats vam veure que a l'hora de calcular quants habitants hi havia per nivell havíem d'afegir referencies nulles per no descontar-nos.

Arbres-R

Justificació de disseny

Per a representar els arbres R, hem implementat una interfície a la que hem anomenat node, i que implementaran els dos tipus de node que poden aparèixer en aquests tipus de estructura de dades, les àrees rectangulars i els nodes fulla que en aquest cas són representats com a instàncies de la classe Monster. Al fer la implementació d'aquesta manera, podem abstraure la classe específica a l'hora de fer una inserció o una eliminació dins un rectangle, doncs tant rectangles com nodes fulla tindran un comportament molt similar en aquestes funcions.

Per a fer la representació modificable, hem fet que l'ordre dels rectangles, és a dir, la quantitat de nodes fill que poden tenir, és pugui modificar des d'una variable.

Per a poder situar els rectangles en el pla xy, aquests guarden un array amb dos Point que representen els dos punts més allunyats, a partir dels quals podem saber dades com el perímetre i l'àrea del rectangle. Aquesta classe Point l'hem creada nosaltres en comptes d'utilitzar la que proporciona java, ja que necessitàvem treballar amb float en comptes de enters i només necessitàvem guardar les coordenades. Els nodes fulla també utilitzen una instància d'aquesta classe per a guardar les seves coordenades.

Explicació d'algorismes i anàlisi de resultats

Inserció de monstres

La inserció de monstres es basa en una inserció típica d'arbres R. Una característica que destaca d'aquesta inserció es la facilitat que te d'auto-equilibrar-se a diferencia dels arbres binaris on s'ha d'aplicar una estratègia concreta com els arbres AVL.

Per inserir un monstre s'ha d'arribar al node fulla dels rectangles corresponent. Aquest procés s'ha implementat de forma recursiva per un motiu destacable. Es important arribar aquest node fulla deixant un rastre dels nodes pels que s'ha passat, el que seria la pujada recursiva. Aquest rastre es crucial ja que cada crida recursiva primer avaluarà si es troba en el node fulla. En cas que es torbi en aquest node fulla inserirà el monstre independentment del overflow, ja que aquesta condició es el que s'avaluarà en la pujada. En el cas que el node actual no sigui fulla es seguirà baixant recursivament en el rectangle on millor encaixi bastant-se en el possible nou tamany del rectangle suposant que el monstre fos fill seu.

Com es pot notar fins al moment només s'està tenint en compte de buscar un node fill adequat i inserir-li el monstre però això no permet balancejar correctament l'arbre. Es aquí on juga un paper essencial la pujada recursiva. En la part de la pujada recursiva, el codi posterior a la crida recursiva, s'encarrega d'evaluar si un rectangle te overflow, en el nostre cas si el número d'elements d'un rectangle es superior a 3 (encara que en el codi implementat aquest ordre es pot canviar de forma dinàmica a partir d'una variable global). En aquest cas es procedeix a fer un resize, el qual consisteix en buscar els dos nodes més llunyans del rectangle amb una cerca iterativa de dos bucles, el que te un cost quadràtic basat en el número de nodes que tingui un rectangle. Un cop obtinguts aquests dos nodes es creen dos nous rectangles i s'insereix a cada un dels dos nodes trobats anteriorment. Es

mira en quin dels dos rectangles encaixa millor cada un dels nodes restants que falten per inserir i es substitueix en el node pare el rectangle que tenia overflow per aquests dos nous.

Un altre cop es pot veure que aquesta adició de nodes no te en compte el overflow, però com ja s'ha comentat la avantatja d'aquesta pujada recursiva es que aquest overflow es controla després de fer una inserció i no abans.

Eliminació de monstres

Per a treure un monstre de l'arbre R, el primer que fem és mirar si el node que estem mirant és fulla o no. Si no ho és seguim buscant dintre dels nodes fill de forma recursiva fins a trobar el punt que busquem. Un cop trobat ens assegurem que és el que estàvem buscant, l'eliminem i tornem a calcular els vèrtex del rectangle que el contenia. En la pujada recursiva mirem si el rectangle que ha perdut el monstre té prous monstres segons l'indicat per la fórmula:

Ordre del rectangle * 0.3

Si el rectangle té un nombre inferior de fills que el resultat d'aquest càlcul s'elimina el rectangle, s'actualitzen els vèrtex el rectangle pare i es tornen a inserir tots els nodes que han quedat orfes.

Cerca de monstres

A l'hora de buscar monstres en una àrea delimitada caldrà considerar quins rectangles tenen part de la seva àrea dins l'àrea de cerca. La idea es anar recorrent tots aquells rectangles que compleixin aquesta condició i descartant tots aquells que no la compleixin aplicant així una poda. Aquest recorregut es farà de forma recursiva baixant per cada rectangle vàlid del node aplicant d'aquesta forma la poda i eliminant totes les branques on des d'un nivell ja superior no es compleixi el criteri.

Ara bé, un punt crucial per aquest apartat es com validar el criteri. Per això s'han plantejant 4 casos possibles.

- Un primer cas on mínim un vèrtex del node esta dins l'àrea de cerca el que ens permet assumir que l'àrea del node i la de cerca interjeccional.

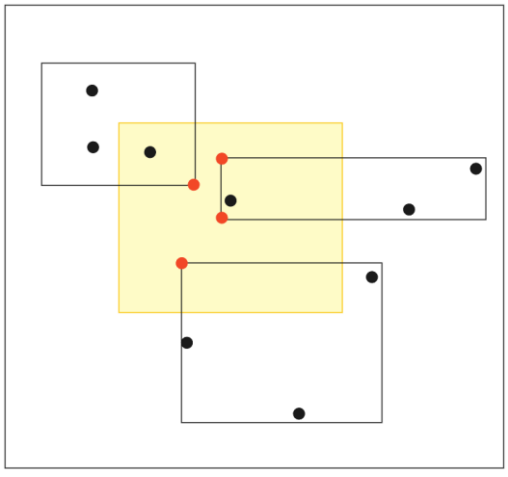


Figura 7: Al menys un vèrtex dins el rectangle de cerca

- Com a cas complementari al mencionat anteriorment es trobaria la situació en que es al menys un dels vèrtex del rectangle de cerca el que està dins del node.

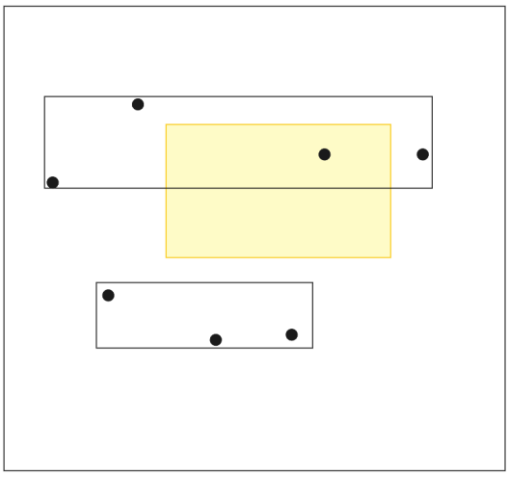


Figura 8: Al menys un vèrtex del rectangle de cerca dins el node

- Com a últim escenari que verifica la condició trobem el cas en que dos dels segments paral·lels del node estiguin fora del rectangle de cerca però els altres dos es trobin dins.

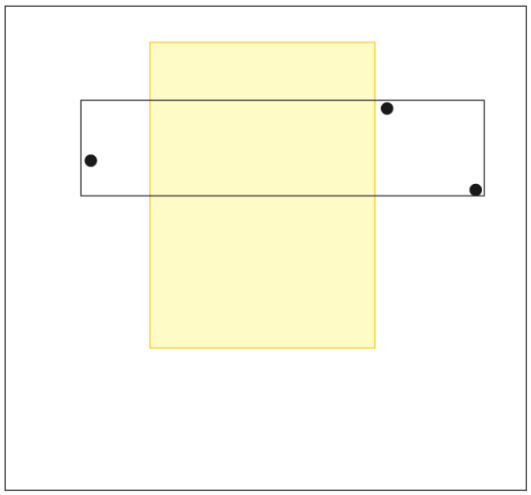


Figura 9: Dos segments paral·lels fora i els altres dos dins

- L'últim escenari es el restant, el qual podem deduir per descart. En aquest cas ningun dels rectangles te cap coincidència i es el que invalida la condició.

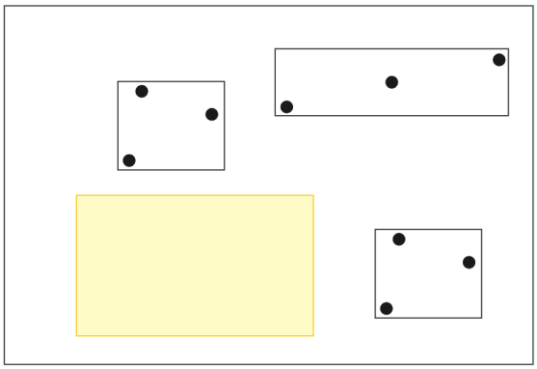


Figura 10: No hi ha cap coincidència

Com es pot observar en la figura 10, el fet que es compleixi aquesta condició d'intersecció no es suficient per afirmar que el monstre es troba dins del rectangle. En aquest cas queda reflectit que s'ha de mirar fins al node fulla dels rectangles i mirar un a un els monstres que conte aquell node si cauen dins l'àrea introduïda.

Zona segura – kNN

L'algoritme kNN té com a objectiu trobar els k punts més propers a un punt donat, independentment de si es troben en un rectangle diferent. En lloc d'afegir el punt i començar la cerca a partir del rectangle al qual s'ha inserit, s'ha optat per iniciar la cerca directament des de l'arrel, comparant les distàncies del punt proporcionat amb els nodes que es van visitant. Aquesta decisió s'ha pres perquè, tot i que això implica recórrer l'arbre una vegada (i probablement no completament, gràcies a les podes), inserir el punt requeriria recórrer l'arbre almenys tres vegades: una per inserir-lo, una altra per trobar el rectangle on s'ha inserit, i una altra per eliminar-lo, sense comptar amb la possibilitat de necessitat de redimensionament. El mètode *kNearestNeighbors* inicia la cerca dels k monstres més propers des de l'arrel de l'arbre. Utilitza una *PriorityQueue* per mantenir els monstres trobats ordenats per distància, amb la distància més gran al capdavant. Això permet que quan la cua conté més de k elements, es pugui eliminar ràpidament el més llunyà. Es crida findNearestNeighbors, que comença la cerca recursiva des de l'arrel. Primer es calcula la distància mínima entre el node actual (monstre o rectangle) i el punt. Seguidament mirem si podem podar: si ja s'han trobat k monstres i la distància mínima del node actual és més gran que la distància més gran en la cua, el node es descarta. Si el node és un rectangle, es continua la cerca en els seus fills. Si el node és un monstre i és desconegut, s'afegeix a la cua si hi ha espai o si és més proper que el més llunyà actualment a la cua. Després de trobar els k monstres més propers, es calculen els valors mitjans de RGB dels colors representant el nivell de perillositat. Aquests valors es converteixen a un codi de color hexadecimal per obtenir el nivell mitjà de perillositat.

Durant l'anàlisi de costos de l'algorisme, s'ha detectat un error important en la poda. Inicialment, el nombre de monstres considerat per a la poda estava fixat en 5, independentment del valor de k proporcionat. Això podria causar que l'algorisme

desestimés monstres vàlids. La correcció implica assegurar-se que la poda es realitza basant-se en el nombre de monstres k especificat.

Anàlisi de costos

La inserció de monstres es basa en una inserció típica d'arbres R, que té la capacitat d'auto-equilibrar-se. Per inserir un monstre, s'ha d'arribar al node fulla dels rectangles corresponent. La cerca del node fulla implica una recursió fins arribar al nivell més baix de l'arbre. En el pitjor dels casos, aquesta operació té un cost O(logN), on N és el nombre total de nodes en l'arbre.

Un cop al node fulla, s'insereix el monstre. Si el node fulla té un overflow, es fa una redistribució dels nodes, el que pot implicar una sèrie d'operacions de divisió de nodes amb un cost de O(M^2), on M és el nombre màxim de fills d'un node. La redistribució implica buscar els dos nodes més llunyans, crear dos nous rectangles, i distribuir els nodes restants entre aquests dos nous rectangles de manera òptima. Aquesta operació té un cost quadràtic degut a les comparacions necessàries per trobar els nodes més llunyans i per distribuir els nodes restants.

Per a treure un monstre de l'arbre R, el primer que fem és mirar si el node que estem mirant és fulla o no. Si no ho és, seguim buscant dintre dels nodes fill de forma recursiva fins a trobar el punt que busquem. La cerca recursiva del monstre a eliminar té un cost de O(log N). Un cop trobat el monstre, s'elimina i es recalculen els vèrtex del rectangle que el contenia. Si el rectangle resultant té menys elements que el mínim permès, es procedeix a una operació de merge o redistribució dels nodes, amb un cost de O(M). Això implica tornar a inserir els nodes orfes al node arrel i recalcular els vèrtex dels rectangles afectats.

El mètode showMonstersInArea recorre l'arbre de manera recursiva. La complexitat d'aquest recorregut depèn del nombre de nodes que compleixen la condició d'intersecció.

Si s'aplica el criteri als rectangles, el que hem implementat, cada node interior es comprova una sola vegada, i si no intersecciona amb l'àrea de cerca, tots els seus fills es descarten d'una vegada. Això redueix el nombre de comparacions necessàries a O(log N) en el millor dels casos, on N és el nombre total de nodes.

Si el criteri s'aplica directament als monstres, cada monstre es comprova individualment, donant lloc a una complexitat de O(N) en el pitjor dels casos, on N és el nombre total de monstres. La Figura 11 presenta una gràfica comparativa entre aquests dos enfocaments, destacant les diferències en eficiencia.

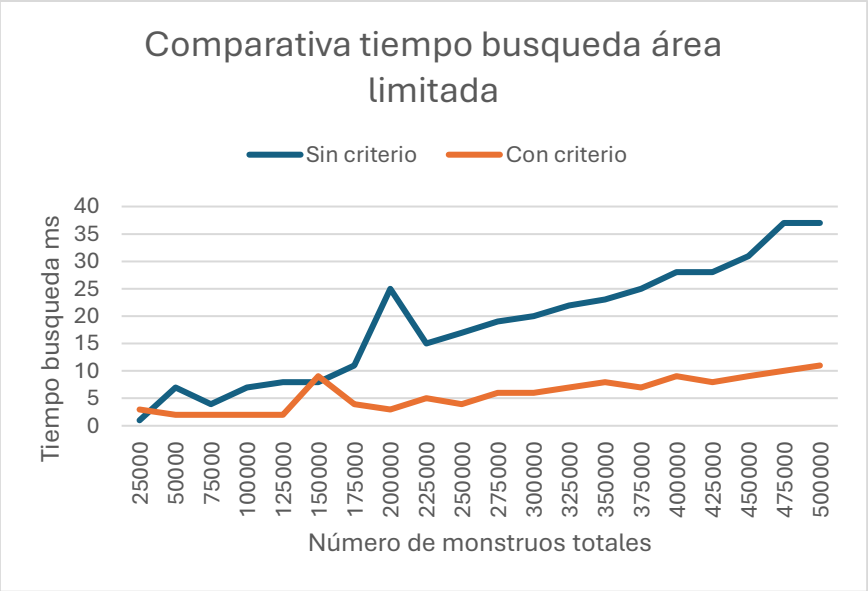


Figura 11: Gràfica comparativa entre aplicar criteri en els rectangles o només en els monstres

Com podem observar, comprovar des del nivell dels rectangles permet eliminar grans subconjunts de monstres amb una sola comprovació reduint significativament el nombre de comparacions necessàries. En el segon enfocament cada monstre es comprova individualment per veure si intersecciona amb l'àrea de cerca, cosa molt ineficient com es pot veure en la gràfica blava, que mostra un creixement lineal indicant que el nombre de comparacions augmenta directament amb el nombre de monstres. Això es deu al fet que cada monstre es comprova individualment sense aprofitar l'estructura jeràrquica de l'arbre R.

En conclusió la comparativa presentada a la Figura 11 demostra la importància d'aplicar el criteri de cerca als rectangles en lloc de fer-ho directament als monstres. Aquest enfocament permet una poda més eficient reduint significativament el nombre de comparacions necessàries i millorant així la velocitat i l'eficiència global de l'algoritme de cerca.

L'algoritme kNN implementat en la classe *RTreeKnn* té diverses etape, cadascuna amb el seu propi cost computacional. La inicialització de la PriorityQueue amb capacitat k és una operació de cost constant, O(1). El mètode kNearestNeighbors coordina la cerca dels veïns més propers i el càlcul del nivell mitjà de perillositat. La complexitat d'aquest mètode es divideix en diverses parts, incloent la crida a findNearestNeighbors que és la part principal de la cerca recursiva.

La funció findNearestNeighbors realitza la cerca recursiva en l'arbre R. La complexitat d'aquesta funció depèn de diversos factors. Calcular la distància mínima entre el node i el punt té un cost constant, O(1). La poda basada en la comparació de distàncies també és O(1). Si el node és un rectangle, la funció crida recursivament a tots els seus fills. Això vol dir que en el pitjor dels casos es recorreran tots els nodes de l'arbre, amb un cost O(N), on N és el nombre total de nodes en l'arbre. Si el node és un monstre desconegut i la cua té menys de k elements, o la cua està plena i el el monstre el monstre si és més proper que el més llunyà en la cua, aquest es pot afegir a la cua en O(log k) temps.

Després de trobar els k monstres més propers, es calcula el nivell mitjà de perillositat. Per a cada monstre en la PriorityQueue, es decodifica el color del nivell de perillositat i es sumen els valors RGB. Decodificar el color i accedir als valors RGB és una operació constant, O(1), però com que cal fer-ho per cada un dels k monstres, el cost total és de O(k). Calcular les mitjanes dels valors RGB també té un cost O(k). Els resultats es reordenen en llista inversa per mantenir l'ordre de proximitat correcte. Aquesta operació té un cost lineal respecte a k, O(k).

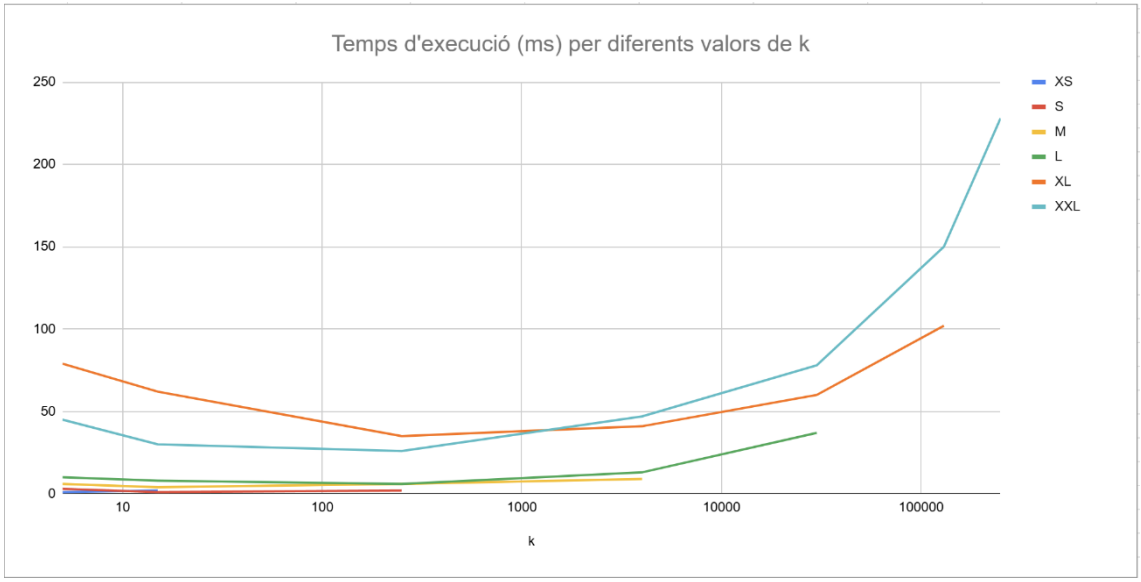


Figura 12: Comparació de cost de temps d'execució de l'algorisme kNN per als diferents dataset amb diferents valors de k.

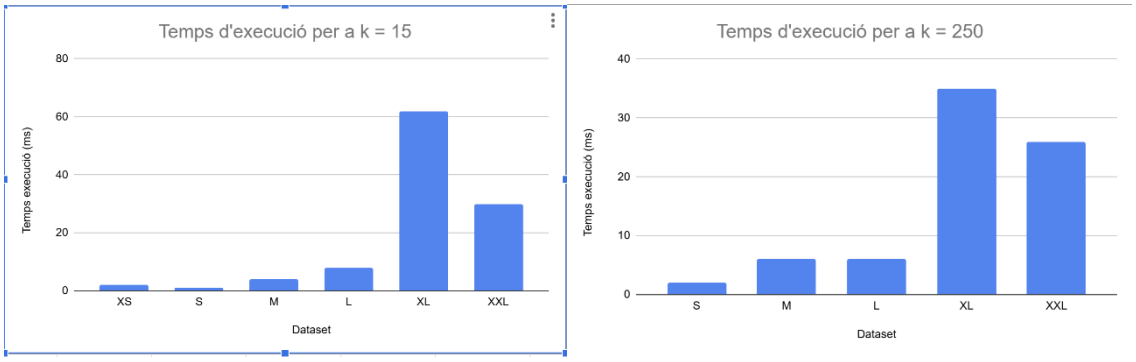
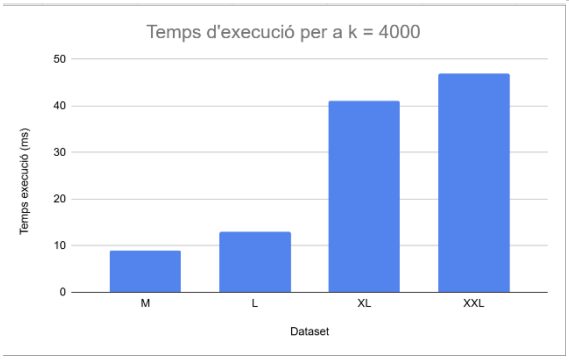


Figura 13 i 14: Comparació de cost de temps d'execució de l'algorisme kNN per a k = 15 i k = 250 amb datasests petits, mitjans i grans.



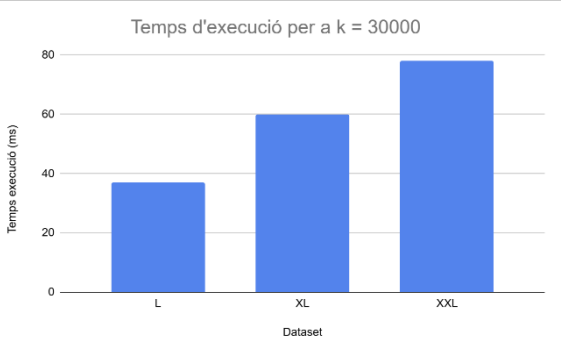


Figura 15 i 16: Comparació de cost de temps d'execució de l'algorisme kNN per a k = 4000 i k = 30000 amb datasests mitjans i grans.

Com podem observar, els temps d'execució augmenten amb el valor de k i el tamany dels datasets. Això és esperable, ja que trobar més veïns requereix més comparacions i costos de càlcul. La diferència és més notable en els tamanys de datasets més grans (XL i XXL). Concretament, a les figures 13 i 14 és fàcilment reconeixible com l'algoritme no escala bé.

A més, podem observar a partir de la Figura 12 que la corba que representa el cost d'execució en els datasests grans primer disminueix fins a més o menys arribar al punt de k = 250 per després tornar a augmentar. Això podria indicar que per a valors de k molt petits, l'algoritme tampoc és gaire eficient. Això pot ser perquè, en haver de trobar els k veïns més propers, si aquesta k és molt petita, l'algoritme es pot veure molt afectat per la variabilitat en la distribució dels punts del dataset.

En conclusió, k representa un aspecte rellevant a estudiar quan es vol aplicar aquest algorisme. Seleccionar el valor òptim de k és essencial per equilibrar la variància i per garantir un rendiment adequat de l'algoritme en funció del conjunt de dades.

Explicació dels mètodes de proves

Prèviament a la implementació de la visualització, l'única manera de comprovar si els algorismes funcionaven correctament era amb l'eina de *dubugging*. Una vegada dita funcionalitat va ser implementada, es van poder comprovar de manera més exhaustiva els algorismes implementats.

Problemes observats

Els principals problemes observats es van trobar a la implementació d'inserció. Inicialment, a cada nivell de la pujada recursiva controlàvem l'*overflow* dels nodes fills però no actualitzàvem l'àrea ocupada pel rectangle pare en cas de *resize*. Una vegada es va tenir en consideració aquest cas va sorgir un altre conflicte. Concretament, quan a la pujada recursiva s'actualitzava l'àrea d'abast d'un rectangle, també s'alteraven les àrees de rectangles fills. Després d'analitzar i debuggar el codi, es va identificar el problema; el mètode que retornava els vèrtex d'un rectangle retornava la referència dels vèrtex en comptes d'una copia. Degut a que per a recalcular els vèrtex d'un rectangle pare s'havien de tractar i treballar amb els vèrtex dels rectangles fills, s'alteraven les àrees dels fills quan l'objectiu era únicament actualitzar el pare.

Taules

Justificació de disseny

La taula està dissenyada per ser una estructura de dades genèrica, la qual cosa permet que sigui utilitzada amb qualsevol tipus d'objecte. Aquesta flexibilitat és essencial per assegurar que la taula pot ser adaptada a diferents contextos i tipus de dades.

La taula hash s'inicialitza amb una mida inicial proporcional a la mida del conjunt de dades (datasetSize). Concretament, s'utilitza el 20% de datasetSize per calcular la mida inicial de la taula. Aquesta decisió es pren per trobar un equilibri entre l'espai de memòria utilitzat i la reducció de col·lisions.

El factor de càrrega (loadFactor) es defineix en 0.75. Aquest valor és un compromís comú en la majoria d'implementacions de taules hash, ja que proporciona un bon equilibri entre la velocitat de cerca i la utilització de memòria. Quan la taula excedeix aquest factor de càrrega, es redimensiona per mantenir l'eficiència.

S'ha establert una mida màxima de 5 per a les llistes de sinònims. Això es fa per limitar el nombre d'elements que poden estar en cada bucket i així reduir el temps de cerca dins de cada llista, mantenint la complexitat propera a O(1) en la majoria dels casos.

El rehashing es realitza quan es detecta una col·lisió i la mida de la llista de sinònims excedeix el límit establert. S'utilitza una segona funció de hash per redistribuir els elements en la taula (double hashing). Si els intents de rehashing arriben al màxim permès (maxRehashAttempts), es redimensiona la taula.

La taula utilitza dues funcions de hash per gestionar col·lisions. La primera funció de hash es basa en el mètode de hash de Horner, utilitzant una base primer (31) per calcular el hash inicial. La segona funció de hash utilitza una base diferent (37) per proporcionar un desplaçament en cas de col·lisió (double hashing). Aquestes funcions es van escollir per la seva eficàcia i distribució uniforme dels valors hash.

La supressió d'elements també està gestionada, assegurant que la taula es redimensiona a una mida més petita si la proporció d'elements cau per sota del 50%. Això ajuda a optimitzar l'ús de memòria i mantenir l'eficiència de la taula hash.

Explicació d'algorismes

Eliminació d'invents

Per l'eliminació d'invents es fa una cerca de l'element a retirar. Es troba la seva posició dins la taula aplicant el criteri de cerca i s'elimina de la llista de sinònims d'aquella posició.

Al treure l'element s'ha de vigilar si el número d'elements de la taula ha sigut modificat, cosa que passarà en cas que la llista de sinònims hagi quedat buida. Si el disminueix el número d'elements caldrà comprovar si s'està per sota el llindar d'elements mínim. Es molt important fer aquesta comprovació per no desperdiciar memòria ja que sinó podria passar que s'eliminin molts elements i el nombre de caselles lliures sigui molt superior a les caselles plenes i que per lo tant s'estigui desaprofitant molt espai. Per això s'ha considerar un valor raonable reduir el tamany de la taula sempre que el número d'elements sigui inferior a la meitat de la taula.

Cada cop que es faci un resize s'hauran d'inserir els elements de la taula en la nova taula reduïda.

Estadística de colors

Com fer els càlculs que demana aquesta funcionalitat de cop, és a dir, un cop totes les dades han estat inserides a la taula de hash requeriria fer un recorregut d'aquesta amb cost N, el que hem acabat fent és tenir una sèrie de comptadors a la classe que representa el canal ciència que es van actualitzant en fer qualsevol operació que pugui afectar a si els invents estan o no trencats. D'aquesta manera quan l'usuari vol veure les estadístiques només cal llegir aquestes dades i fer les operacions pertinents, tot mostrant-les per pantalla.

Fer inventari

Per implementar la cerca d'un invent pel seu nom, s'ha creat el mètode get a la nostra HashTable. Primer, es calcula el primer hash a partir del nom proporcionat, obtenint així la posició inicial a la taula on es podria trobar l'invent. Donat que l'invent pot no estar a la

posició indicada pel primer hash degut a col·lisions, es recorre la taula en un bucle a partir d'aquesta posició, comprovant si algun dels elements coincideix amb el nom buscat.

Si no es troba l'invent en la posició inicial, per a cada iteració es calcula una nova posició utilitzant el segon valor de hash i un el nombre d'intents realitzats fins al moment. Aquest procés es repeteix fins a un màxim de maxRehashAttempts. Si després d'aquests intents l'invent no ha estat trobat, es retorna null ja que voldrà dir que no existeix aquest valor a la taula.

Aquest mètode és força eficient ja que en el millor dels casos el cost és O(1) i en el pitjor O(maxRehashAttempts).

Cerca alfabètica

L'algoritme searchByLetter, que pretén solucionar el problema de la cerca alfabètica, realitza una cerca d'invencions que comencen per una lletra específica i retorna la llista d'invencions ordenada alfabèticament. Es recuperen totes les invencions amb la funció getAll implementada a HashTable, i es recorre la llista verificant si el nom de cada invenció comença per la lletra especificada. Si és així, l'invenció s'afegeix a una llista de resultats. Finalment, s'ordena alfabèticament la llista fent ús de Collections.

El mètode getAll s'ha implementat recorrent totes les llistes de la taula hash i afegint tots els elements a una única llista. Ràpidament ens adonem que això no és gaire òptim a nivell de recursos. El mètode getAll() té una complexitat de O(n), sent n el número d'elements a la taula de hash. A més, searchByLetter recorre aquesta llista, per la qual cosa es suma un altre O(n). Finalment, ordenar la llista amb Collections té una complexitat O(m*log*m), sent m el número d'invencions de la llista de resultats.

Tafaneria

Es recupera l'invent amb un get de HashTable i posteriorment si no estaba trencat es marca com a tal i es retorna true. En cas contrari, es retorna false.

Això torna a representar un cost força baix ja que el cost de get ja sabem que pot ser entre O(1) i O(maxRehashAttempts) i el demés presenta un cost O(1).

Reparació

Anàlisi de resultats

L'anàlisi d'aquest apartat es centrarà principalment en comparar i extreure conclusions de la gestió d'espai i els seus costos més que del cost operacional i temporal. Les taules es centren en les cerques per aconseguir costos molt baixos per això s'ha considerat de poca rellevància analitzar aquesta part.

Per un altre banda durant la implementació d'aquesta estructura de dades si que s'ha vist que els costos de memòria son més fràgils i s'han de considerar diferents casuístiques en la seva creació per tal de no malbaratar-ne molta.

Un primer punt que es vol comentar es com gestionar el número de posicions ocupades en les taules utilitzant llistes de sinònims. Això ha sigut un punt conflictiu ja que en un principi es va decidir que es comptaria una posició ocupada quan la llista de sinònims estigues plena. Però posteriorment es va veure que això impedia fer una eliminació de forma coherent ja que si per exemple es tenen 2 arrays amb llistes de fins a 5 sinònims si 1 llista està plena i l'altre té 3 elements al fer un remove de la llista de 5 passarà a marcar com que hi ha un element menys fent que el número d'elements sigui 0.

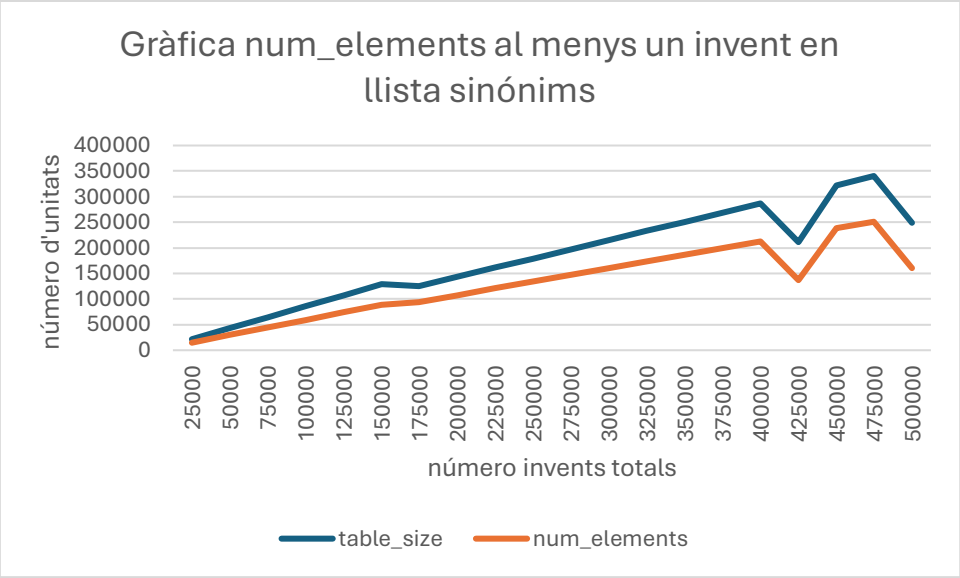
Aquesta situació implica que s'haurà de fer un resize per reduir i com que es tindrà una taula de tamany menor al número d'invents implicarà que s'ha de fer un resize per augmentar. Si ve es cert que això es un cas poc probable amb taules de molts elements en el seu moment es va veure un criteri a tenir en compte i pel que es va optar.

Despès d'haver-se fet l'anàlisi que es pot veure a continuació s'ha conclòs que el criteri no agafat hagués sigut millor. S'ha vist que el cost extra que podria haver-hi en aquells casos val la pena en comparació al que es guanya. Per una banda, com es veurà en les comparacions els costos de memòria son notablement majors. Per l'altre banda es pot assumir que per datasets grans el problema que suposava l'eliminació, considerant que una posició esta ocupada només quan la llista esta plena, fent que es passí del llindar d'ocupació mínima tenint més invents que el nou tamany de la taula, serà un cas molt improbable de succeir i el cost de resize que tindrà no suposa un cost asimptòtic de grau major.

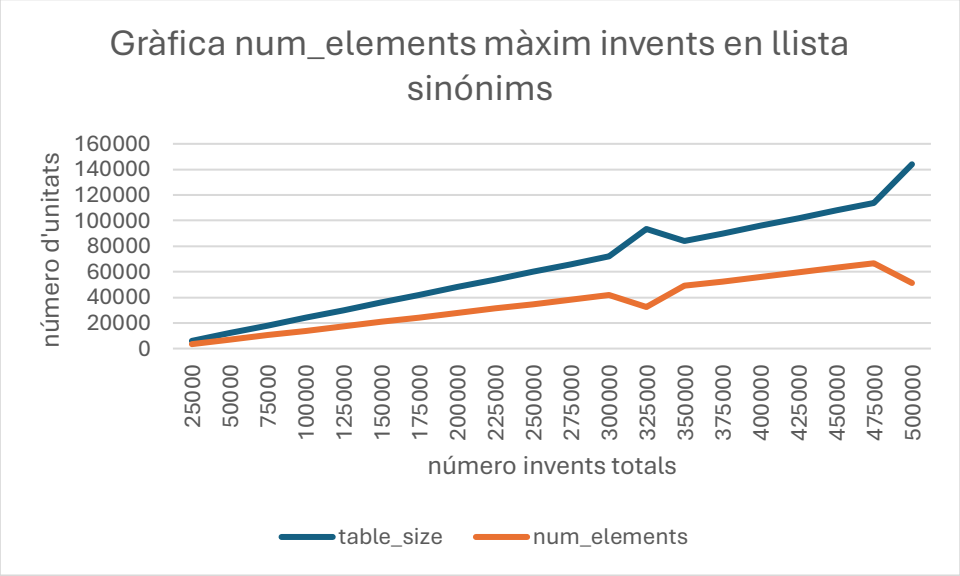
En aquestes dos primeres taules es pot veure una comparació del tamany de taula i el número d'elements que té, on el número d'elements son posicions ocupades, en base al número total d'invents per cada un dels dos criteris seguits.

La primera taula fa referencia a considerar que un element equival a que una llista de sinònims tingui almenys un invent. La segona es el seu oposat fent que es consideri un element com a una llista plena. La línia taronja fa referencia al número d'elements mentre que la blava al tamany de la taula.

Veiem que el tamany per datasets als arriba a ser fins al doble en la primera taula el que deixa reflectida i serveix com a argument per defensar la conclusió comentada anteriorment.



Il·lustració 5: Gràfica número elements en base a minim un invent en la llista de sinonims

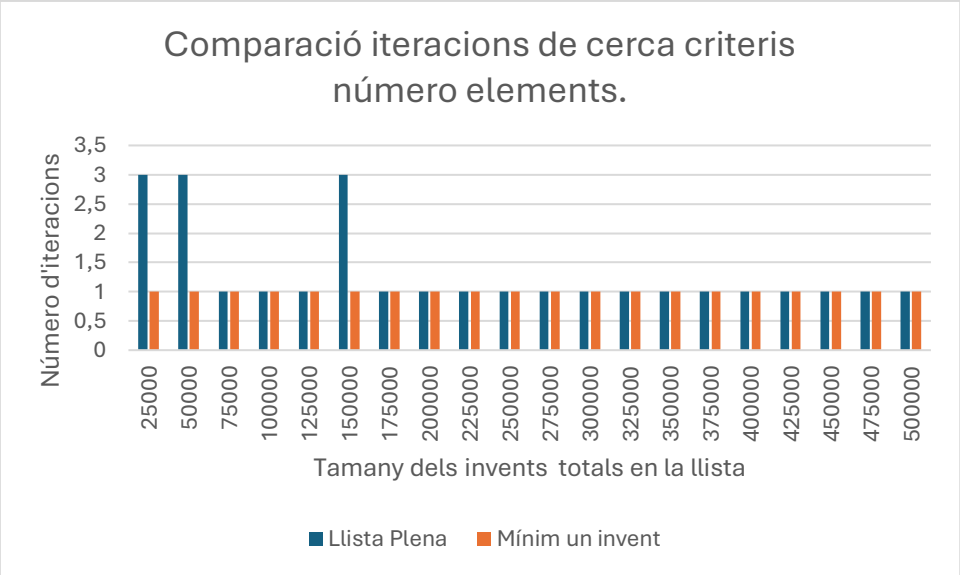


Il·lustració 6: Gràfica número elements en base llista de sinonims plenes

Tot i que ja s'ha vist clarament les avantatges que ofereix considerar llistes plenes contra llistes amb mínim un element, es podria pensar que al tenir més elements les funcions de hash serien més eficaces, ja que al obtenir el mòdul amb un rang de valors major, seria més improbable accedir a una posició ja ocupada. Per assegurar aquesta consideració s'ha fet un analisí de barres comparant el número de hashes que s'havia de fer a un invent per

trobar-lo en la taula. Com es pot veure en la gràfica no hi ha casi ninguna diferencia. Això fa que un altre cop el criteri de llistes plenes sigui millor.

En la gràfica si que es pot veure que es més probable necessitar més d'una hash per taules amb menys elements ja que aquests estan més concentrats però tot i així el cost addicional que suposa es menyspreable.



Il·lustració 7: Taula de barres comparatives per número d'iteracions segons críteri

Un altre criteri en la implementació de les taules es el tamany de les llistes de sinònims. Això es un punt important ja que en les cerques pot suposar un augment de cost considerable. Per aquest motiu, s'ha volgut comparar els tamanys de les taules seguint les llistes utilitzades en el codi, sent una llista de 5 invents, en contra d'una amb un tamany notòriament més gran sent aquesta d'una desena part del tamany del dataset.

En la gràfica es pot veure que hi ha una diferencia de tamany de taula, però aquesta no s'apropa a la comparada prèviament. Si bé es cert que si ens guiéssim per un criteri centrar en la gestió de memòria podria cridar més la atenció tenir llistes més grans s'ha de contemplar que tenint el dataset de 500.000 invents en la gràfica es pot apreciar que la taula on la desena part del dataset son els invents per llista es un 40% més petita que la de 5 sinònims, això a canvi de que cada llista sigui de fins a 10.000 cops més gran. No només

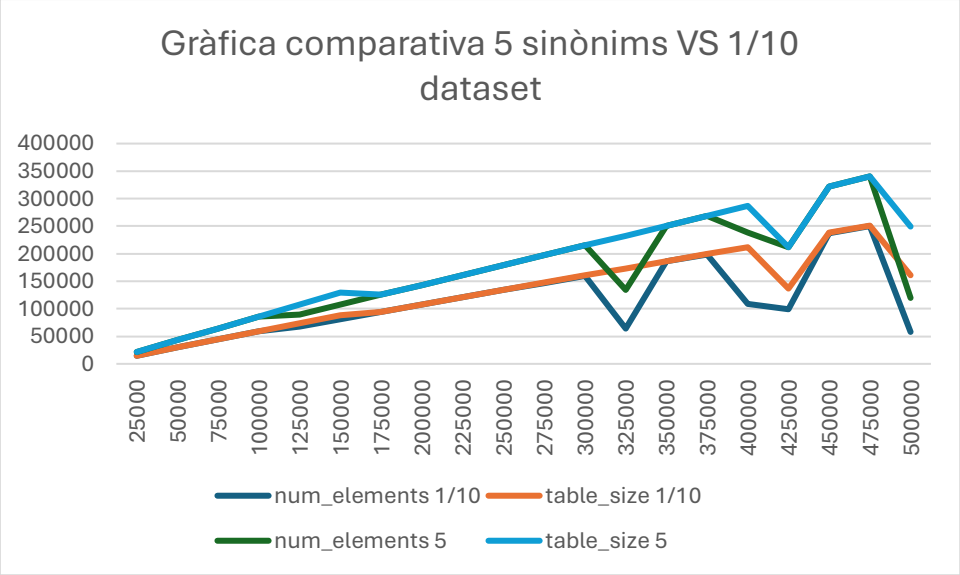
son uns canvis desproporcionats sinó que la taula perd en certa part la seva essència fent que el seu cost de cerca fos molt pròxim a 1 passi a ser:

$$H \cdot \frac{D}{10}$$

En canvi respectant una constant de sinònims veiem que el cost es manté constant

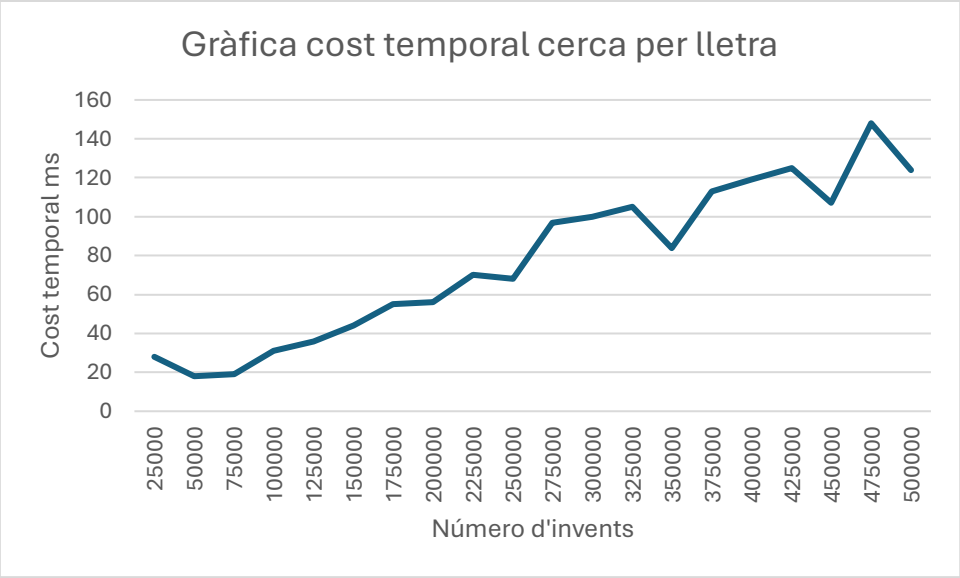
$$H \cdot 5$$

On H es el número de cops que hem de fer el hash i D el número de tamany del dataset.



Illustració 8: Gràfica comparativa llistes de sinonims diferents tamanys

Després d'analitzar els costs de memòria es volia fer un últim anàlisi. Com s'ha comentat les taules no estan enfocades en algorismes per recorre l'estructura sinó per cercar. Amb la següent gràfica es vol deixar constància de la poca utilitzat de les taules per tal de recorre veient que el cost asimptòtic d'aquesta es lineal en contra del cost constant que s'ha obtingut prèviament en les cerques.



Il·lustració 9:Gràfica del cost temporal de fer una cerca per lletra

Explicació dels mètodes de proves

Per a la comprovació de les taules s'han seguit varies metodologies.

La primera i més exacte ha sigut amb l'eina de *debugging* la qual s'ha utilitzat per anar mirant en cada moment els continguts del array i les llistes de sinònims així com el número d'elements. Aquest mètode s'ha executat d'una forma controlada, és a dir, s'introduïen valors escollits a consciencia i es feia un càlcul teòric de quin tindria que ser el resultat i així verificar que el que passava era correcte.

Un altre mètode de comprovació que s'ha utilitzat ha sigut imprimint per pantalla els resultats d'aplicar la funció de hash i així veure el codi obtingut per una entrada. Aquest mètode també s'ha aplicat mostrant el número d'iteracions que necessitava per cercar un element. Amb aquest resultat s'ha pogut veure que efectivament el hash era directe i no es quedava en algun bucle fent més iteracions de les que necessitava.

Problemes observats

Com ve s'ha comentat en l'anàlisi un problema observat era com gestionar el número d'elements que es tenien en l'array juntament amb les llistes de sinònims. Un altre problema destacable ha sigut el fet que alguns valors en el càlcul de hash arribaven a números molt alts, passant el valor màxim que avarca una variable de número enter. Això no s'ha gestionat en el codi ja que s'ha vist el error al redactar la memòria i fer l'anàlisi de costos on al provar amb número de dades molt grans s'ha arribat a aquest valors. Una possible solució a aquest problema podria haver sigut gestionar aquells càlculs que tenien perill amb variables de tipus *long*, ja que aquests càlculs son els que es troben abans d'aplicar el mòdul del tamany de la taula, i un cop aplicat el mòdul i ja de forma segura que el valor obtingut seria vàlid com a *int* guardar-lo com el valor de hash final.